# Sheet 3

## Question 1

1. $T(n) = T(n-1)+5$, $T(1) = 1$ and $n > 1$.

①

1. $T(n) = T(n-1) + 5$

① ~~sub~~ Back substitution

$T(n) = T(n-1) + 5 \rightarrow ①$

$T(n-1) = T(n-2) + 5 \rightarrow ②$

$T(n-2) = T(n-3) + 5 \rightarrow ③$

From ① & ②

$T(n) = T(n-2) + 5 + 5 \rightarrow ④$

From ③ & ④

$T(n) = T(n-3) + 5 + 5 + 5 \rightarrow T(n-3) + 3 \times 5$

at $k$

$T(n) = T(n-k) + 5k$

at $T(1) = 1$

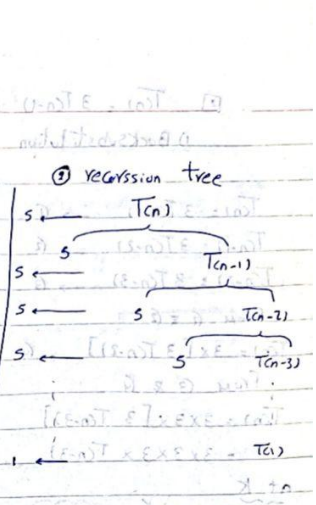
$n - k = 1 \rightarrow k = n-1$

$\therefore T(n) = T(n-n+1) + 5(n-1)$

$= T(1) + 5(n-1)$

$T(n) = O(n)$

② Regression tree




③ Master theory

$T(n) = T(n-1) + 5$

$a = 1$ $\quad k = 0$ $\quad 5n^0$

$b = 1$

$\rightarrow a = 1$ Case 2

$\therefore O(n + P(n)) = O(n^{k+1}) = O(n)$

CamScanner الممسوحة ضوئيا بـ

2. T(n) = 3T(n−1), T(1) = 4 and n > 1

②

$\boxed{2}$ $T(n) = 3T(n-1)$

1) Backsubstitution

$T(n) = 3T(n-1) \rightarrow (1$

$T(n-1) = 3T(n-2) \rightarrow (2$

$T(n-2) = 3T(n-3) \rightarrow (3$

From (1 & (2

$T(n) = 3 \times [3T(n-2)] \rightarrow (4$

From (3 & (4

$T(n) = 3 \times 3 \times [3T(n-3)]$

$= 3 \times 3 \times 3 \times T(n-3)$

at K

$T(n) = 3^K T(n-K) + 1$

$T(1) = 4$

$n-K = 1$

$K = n-1$

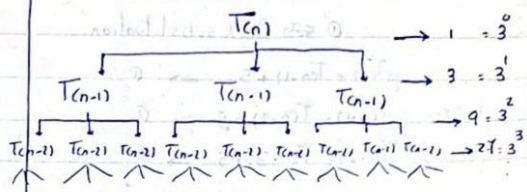
$\therefore T(n) = 3^{n-1} T(n-n+1)$

$= 3^{n-1} \times 4$

~~$= O(n$~~

$O(3^n)$

2) Recursion Tree




$T(n) = 3^n$

$O(3^n)$

3) Master theorem

$T(n) = 3T(n-1) + (0 \times n^1)$

$\hookrightarrow K=1$

$a = 3$

$b = 1$

$\therefore a > 1$ (case 3)

$\therefore$ ~~$O(a^{n/b}) = O(n$~~

$O(a^{n/b} \times n^K) = O(3^n)$

CamScanner الممسوحة ضوئيا بـ

3. T(n) = T(n−1)+n, T(1) = 1 and n > 0.

③

$\boxed{3}$ $T(n) = T(n-1) + n$

1) Back substitution

$T(n) = T(n-1) + n \rightarrow (1$

$T(n-1) = T(n-2) + (n-1) \rightarrow (2$

$T(n-2) = T(n-3) + (n-2) \rightarrow (3$

From (1 & (2

$T(n) = T(n-2) + (n-1) + n \rightarrow (4$

From (3 & (4

$T(n) = T(n-3) + (n-2) + (n-1) + n$

at $k$

$T(n) = T(n-k) + (n-(k-1)) + (n-(k-2)) + \dots + n$

at $n-k=1$

$k = n-1$ ∴

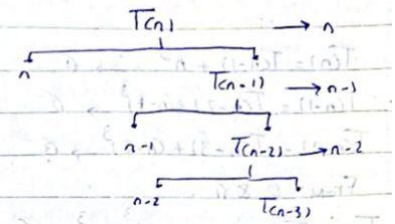
$T(n) = T(n-n+1) + (n-n+1) + (n-n+2) + \dots + n$

$= T(1) + 1 + 2 + 3 + \dots + n$

$= T(1) + \frac{n(n+1)}{2}$

$= O(n^2)$

2) Recurssion tree




$T(1) \rightarrow 1$

$1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$

$O(n^2)$

3) Master theorem

$T(n) = T(n-1) + n$

$a = 1$ $k = 1$

$b = 1$

$a = 1$ Case 2

∴ $O(n^{k+1}) = O(n^2)$

CamScanner الممسوحة ضوئيا بـ

4. $T(n) = T(n/2)+n$, $T(1) = 1$ and $n > 1$

Master theorem :

$a = 1, b = 2, k = 1$ case 3 $O(n)$

5. $T(n) = T(n/3)+0$, $T(1) = 1$ and $n > 1$.

Master theorem :

$a = 1, b = 3, k = 0$ case 2 $O(\log n)$

6. T(n) = T(n−1)+n3, T(1) = 1 and n > 1

④

6) $T(n) = T(n-1) + n^3$

1) Back substitution :

$T(n) = T(n-1) + n^3 \rightarrow (1$

$T(n-1) = T(n-2) + (n-1)^3 \rightarrow (2$

$T(n-2) = T(n-3) + (n-2)^3 \rightarrow (3$

From (2 & (1

$T(n) = T(n-2) + (n-1)^3 + n^3 \rightarrow (4$

From (3 & (4

$T(n) = T(n-3) + (n-2)^3 + (n-1)^3 + n^3$

at K

$T(n) = T(n-K) + (n-(K-1))^3 + (n-(K-2))^3 + \dots + n^3$

at $n-K=1$

$K = n-1$

$\therefore T(n) = T(n-n+1) + (n-n+2)^3 + (n-n+3)^3 + \dots + n^3$

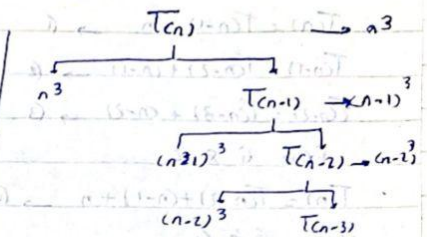
$= T(1) + 2^3 + 3^3 + \dots + n^3$

$= 1 + 2^3 + 3^3 + \dots + n^3$

$= \left(\sum_{i=1}^{n} i\right)^2 = \left(\frac{n(n+1)}{2}\right)^2$

$= O(n^4)$

2) Recursion Tree




$1 + (n-1)^3 + (n-2)^3 + (n-3)^3 + \dots + n^3$

$O(n^4)$

3) Master theorem

$T(n) = T(n-1) + n^3$

$a = 1$ $\quad K = 3$

$b = 1$

$a = 1$ Case 2

$O(n^{K+1}) = O(n^4)$

CamScanner الممسوحة ضوئيا بـ

7. $T(n) = 2T(n/2)+n$, $T(1) = 1$ and $n > 1$

Master theorem :

a = 2, b = 2, k = 1  case 2  O(nlogn)

8. $T(n) = 4T(n/3)+n$, $T(1) = 1$ and $n > 1$

Master theorem :

a = 4, b = 3, k = 1  case 1  O(n^1.908)

9. $T(n) = 2T(n/2)+\log n$, $T(1) = 1$ and $n > 1$

Master theorem :

a = 2, b = 2 case 2  O(n)

10. $T(n) = 2T(n/2)+n2$, $T(1) = 1$ and $n > 1$

Master theorem :

a = 2, b = 2, k = 2  case 3  O(n^2)

11. $T(n) = 2T(n/4)+\sqrt{n}$, $T(1) = 1$ and $n > 1$

Master theorem :

a = 2, b = 4, k = 1/2  case 2  O(sqrt(n) logn)

# Question 2

```
public static int f(int n) {
    if (n == 1) {
        return 1;
    } else {
        return f(n-1) + f(n-1);
    }
}
```

①

Q2

1) recurence relation

$T(n) = T(n-1) + T(n-1) + 1$

$= 2\,T(n-1) + 1$

1) Backsubstitution

$T(n) = 2\,T(n-1) + 1 \longrightarrow (1$

$T(n-1) = 2\,T(n-2) + 1 \longrightarrow (2$

$T(n-2) = 2\,T(n-3) + 1 \longrightarrow (3$

From (2 & (1

$T(n) = 2\,[2\,T(n-2) + 1] + 1 \longrightarrow (4$

From (3 & (4

$T(n) = 2 \times 2\,[2\,T(n-3) + 1] + 2 + 1$

$= 2 \times 2 \times 2\,[T(n-3)] + 2 \times 2 + 2 + 1$

$= 2^3\,T(n-3) + 2^2 + 2^1 + 2^0$

at K

$n - K = 1$

$(K = n-1)$ $T(n) = 2^K\,T(n-K) + 2^{K-1} + 2^{K-2} + \dots + 2^0$

$T(n) = 2^{n-1}\,T(n-n+1) + 2^{n-1-1} + 2^{n-1-2} + \dots + 2^0$

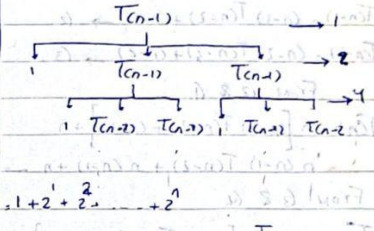
$= 2^{n-1} + 2^{n-2} + 2^{n-3} + 2^{n-4} + \dots + 2^{n-n}$

$= 2^{n-1} + 2^{n-2} + \dots + 2$

$= 2^{n+1} - 1$

$O(2^n)$

2) recurssion tree




$= 1 + 2^1 + 2^2 + \dots + 2^n$

$O(2^n)$

3) Master theorem

$T(n) = 2\,T(n-1) + 1 \rightarrow = n^0$

$a = 2$ $\quad k = 0$

$b = 1$

$a > 1$ $\quad$ case 3

$O(a^{n/b} \cdot n^k) = O(2^n)$

CamScanner الممسوحة ضوئيا بـ

```java
public static int f(int n) {
    int result;
    if (n == 1) {
        result = 1;
    } else {
        result = 0;
        for(int i = 0; i < n; i++) {
            result = result + f(n-1);
        }
    }
}
```

② ①

8 Recurrence Relation

$T(n) = n\,T(n-1) + n$

1) Backsubstitution

$T(n) = n\,T(n-1) + n \rightarrow (1$

$T(n-1) = (n-1)\,T(n-2) + (n-1) \rightarrow (2$

$T(n-2) = (n-2)\,T(n-3) + (n-2) \rightarrow (3$

From (2 & (1

$T(n) = n\left[(n-1)\,T(n-2) + (n-1)\right] + n$

$= n(n-1)\,T(n-2) + n(n-1) + n \rightarrow (4$

From (3 & (4

$T(n) = n(n-1)\left[(n-2)\,T(n-3) + (n-2)\right] + n(n-1) + n$

$= \underbrace{n(n-1)(n-2)}_{n!}\,T(n-3) + n(n-1)(n-2) + n(n-1) + n$

$a + \underbrace{K}$

$T(n) = n(n-(k-2))(n-(k-1))\,T(n-k) + n(n-(k-2))(n-(k-1)) + n(n-(k-2)) + n$

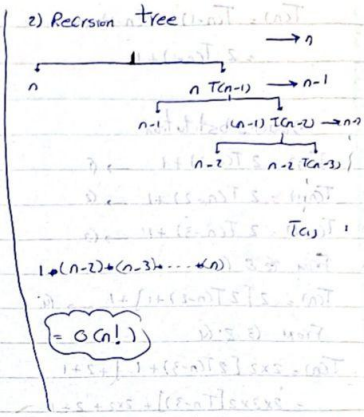
$n-k=1$

$k = n-1$ $\quad T(n) = n(n-n+3) * (n-n+2) * T(1) + n(n-n+3)\cdot(n-n+2) + (n)(n-n+3) + n$

$= n(3)(2) + n(3)(2) + n(3) \ldots + n$

$O(n!)$

2) Recursion tree




$1 + (n-2) + (n-3) + \ldots + (n)$

$= O(n!)$

3) Master theorem

$T(n) = n\,T(n-1) + n$

Failed

Not constant

CamScanner الممسوحة ضوئيا بـ

```java
public static int f(int n) {
    if (n == 1) {
        return 1;
    } else {
        return f(n-1) + f(n-2);
    }
}
```

Fibonacci -> O(2^n)

```java
public static int f(int n) {
    if (n == 1) {
        return 1;
    } else {
        int y =  f(n/2) + f(n/2);
        for(int i = 0; i < n; i++) {
            y = y + 1;
        }
        return y;
    }
}
```

T(n) = 2 T(n/2) + n

Master Theorem :

a = 2, b = 2, f(n) = n ^ log2(2) = n, so k = 1

case 2 -> O(nlogn)

```java
public static int f(int n) {
    if (n == 1) {
        return 1;
    } else {
        int y =  f(n/2) + f(n/2);
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                y = y + 1;
            }
        }
        return y;
    }
}
```

T(n) = 2 T(n/2) + n^2

Master Theorem :

a = 2, b = 2, k = 2

case 3 -> O(n^2)

```java
public static int f(int n) {
    if (n == 1) {
        return 1;
    } else {
        int y =  f(n/2) + f(n/2);
        for(int i = 0; i < n; i++) {
            for(int j = 0; j*j < n; j++) {
                y = y + 1;
            }
        }
        return y;
    }
}
```

T(n) = 2 T(n/2) + n^(3/2)

Master theorem :

a = 2, b = 2, k = 3/2

case 3 -> O(n^(3/2))